

Programação Orientada a Objetos

Sobrecarga de Operadores



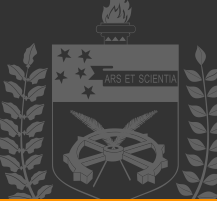
Universidade Federal de Santa Catarina – UFSC

Centro Tecnológico de Joinville – CTJ

Prof. Arthur Gabardo

2026.1

Relembrando...



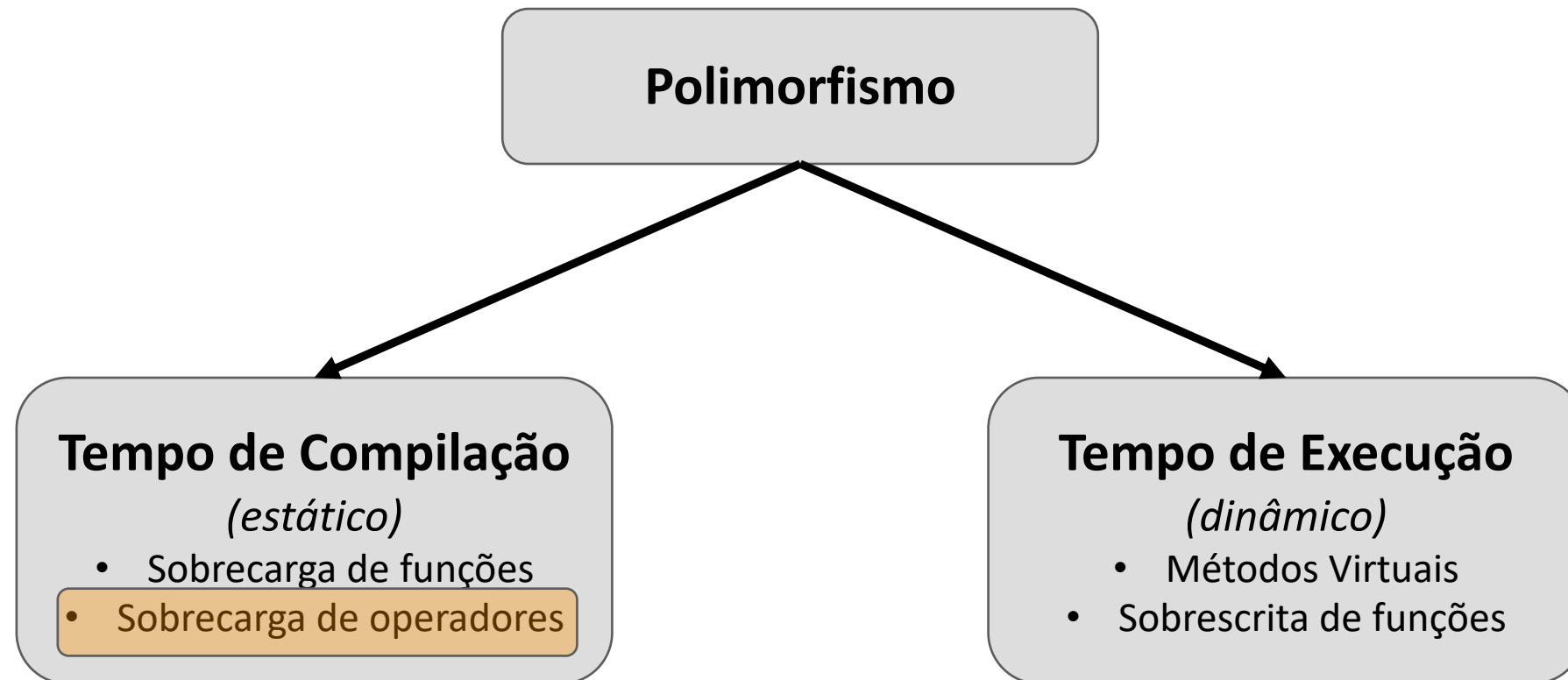
- Herança permite que uma classe estenda os membros de outra
 - Modela uma relação do tipo **é um(a)**
 - Superclasse/Base → Subclasse/Derivada
 - Princípio da Substituição de Liskov
- Associação promove o agrupamento de classes de menor complexidade
 - Modela uma relação do tipo **tem um(a)**
 - Sem palavra-chave ou sintaxe específica

```
class Disciplina {  
    string nome;  
    string codigo;  
    // ...  
};  
  
class Pessoa {  
    protected:  
        string nome;  
    public:  
        Pessoa(string _n) : nome(_n) {}  
    // ...  
};  
  
class Estudante : public Pessoa {  
    int matricula;  
    Disciplina *disciplinas;  
    public:  
        Estudante(string _n, int _m) :  
            Pessoa(_n) {}  
    // ...  
};
```

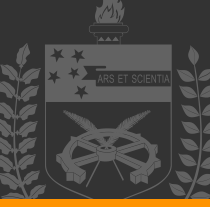
Relembrando...



- Polimorfismo permite que funções tomem diversas formas
 - Em C++ temos dois tipos básicos de polimorfismo



Na aula de hoje



- Operadores
- Sobrecarga de Operadores
- Classes e Funções `friend`

Operadores



- Em C++, operadores são símbolos que representam operações sobre um conjunto de operandos
- Na prática, são funções especiais com sintaxe diferenciada

```
string a = "UFSC ";  
string b = "Joinville";  
  
string c = a + b;  
string d = operator+(a, b);  
  
std::cout << c << std::endl;  
std::cout << d << std::endl;
```

Operadores



Os operadores podem ser:

- Unários → atuam sobre somente um operando

```
int *ptr = &a;  
int result = *ptr + --b - !c
```

- Binários → atuam sobre dois operandos

```
int result = (a[0] + b) * c - (a[1] / b)
```

- Ternários → condicional

```
int result = (a > b) ? a : b;
```



Tipo	Operadores
Aritmético	<code>a+b, a-b, a*b, a/b, a%b, ++a, --a, a++, a--</code>
Relacional	<code>a==b, a!=b, a>b, a>=b, a<b, a<=b</code>
Lógico	<code>!a, a&&b, a b</code>
Atribuição	<code>a=b, a+=b, a-=b, a*=b, a/=b, a&=b, a =b</code>
Bitwise	<code>~a, a&b, a b, a^b, a<<b, a>>b</code>
Ponteiros	<code>*a, &a, a.b, a->b, a[b]</code>
Outros	<code>func(a,b), c ? a : b, sizeof(a), new a, delete a</code>

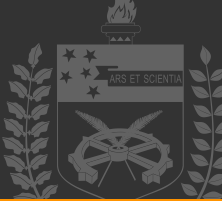
[https://pt.wikipedia.org/wiki/Operadores em C e C%2B%2B](https://pt.wikipedia.org/wiki/Operadores_em_C_e_C%2B%2B)

Sobrecarga de operadores



- Vimos que podemos sobrecarregar funções alterando suas assinaturas
 - Polimorfismo estático (em tempo de compilação)
- Se operadores são funções com uma sintaxe diferenciada
 - Logicamente, podemos também sobrecarrega-los
- A sobrecarga de operadores permite definir o comportamento quando usados em tipos definidos pelo programador (`enum`, `struct`, `class`)
 - Torna a manipulação de objetos mais natural e legível

Sobrecarga de operadores



```
class Ponto3D {
public:
    double x;
    double y;
    double z;
    Ponto3D(double _x, double _y, double _z) :
        x(_x), y(_y), z(_z) {}
    Ponto3D adicionar(const Ponto3D &p) {
        return Ponto3D(x+p.x, y+p.y, z+p.z);
    }
};

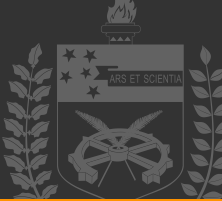
int main() {
    Ponto3D a(0,0,0), b(2,4,1);
    Ponto3D c = a.adicionar(b);
    Ponto3D d = c + b;
    ///! N existe operador Ponto3D + Ponto3D
    return 0;
}
```

```
% g++ add.cpp -o add
```

```
add.cpp:21:19: error: invalid operands to binary expression ('Ponto3D' and 'Ponto3D')
```

```
21 |     Ponto3D d = c + b;
    |                      ~ ^ ~
```

Sobrecarga de operadores

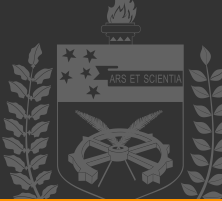


```
class Ponto3D {
public:
    double x;
    double y;
    double z;
    Ponto3D(double _x, double _y, double _z) :
        x(_x), y(_y), z(_z) {}

    // Sobrecarga do operador +
    Ponto3D operator+(const Ponto3D &p) {
        return Ponto3D(x+p.x, y+p.y, z+p.z);
    }
};
```

- Ao sobrecarregar um operador declaramos uma função
 - `<tipo_retorno> operador<operador> (parametros) {...}`
- A função de sobrecarga pode ser membro da classe ou global

Sobrecarga de operadores



```
class Ponto3D {  
public:  
    double x;  
    double y;  
    double z;  
    Ponto3D(double _x, double _y, double _z) :  
        x(_x), y(_y), z(_z) {}  
};  
  
Ponto3D operator+(const Ponto3D &p1, const Ponto3D &p2) {  
    return Ponto3D(p1.x+p2.x, p1.y+p2.y, p1.z+p2.z);  
}
```

- O número de parâmetros original do operador deve ser mantido
 - Unários → um parâmetro, Binários → dois parâmetros
- Quando definimos como função membro um dos parâmetros é implícito (próprio objeto)

Sobrecarga de operadores



Algumas boas práticas ao sobrecarregar operadores:

- Somente use a sobrecarga se o seu significado for claro e adequado
- Mantenha a semântica do operador
 - Seu comportamento deve sempre se manter o esperado
- Ofereça sempre todas as operações relacionadas
 - Se suporta $a+b$, então é esperado que haja $a+=b$ também
 - Se suporta incremento pré-fixado $++a$, deve permitir pós-incremento $a++$
 - Se possui um construtor de cópia, pode-se esperar a sobrecarga de $=$

Exemplos



Operador de acesso

```
class Ponto3D {
public:
    double x, y, z;
    Ponto3D(double _x = 0, double _y = 0, double _z = 0) :
        x(_x), y(_y), z(_z) {}

    // Operador de acesso [] que permite alteracao
    double& operator[](int idx) {
        if (idx == 0) return x;
        if (idx == 1) return y;
        if (idx == 2) return z;
        throw std::out_of_range("Índice inválido para Ponto3D");
    }

    // Operador de acesso [] que nao permite alteracao
    const double& operator[](int idx) const {
        if (idx == 0) return x;
        if (idx == 1) return y;
        if (idx == 2) return z;
        throw std::out_of_range("Índice inválido para Ponto3D");
    }
    // ...
}
```

Exemplos



Operadores de atribuição

```
class Ponto3D {
public:
    // ... membros da classe Ponto3D

    Ponto3D& operator+=(const Ponto3D& other) {
        x += other.x; y += other.y; z += other.z;
        return *this;
    }

    Ponto3D& operator-=(const Ponto3D& other) {
        x -= other.x; y -= other.y; z -= other.z;
        return *this;
    }

    Ponto3D& operator*=(double escalar) {
        x *= escalar; y *= escalar; z *= escalar;
        return *this;
    }

    Ponto3D& operator=(const Ponto3D& other) {
        if (this != &other) {
            x = other.x; y = other.y; z = other.z;
        }
        return *this;
    } // ...
}
```



“Se uma classe precisa de um destrutor, um construtor de cópia, ou um operador de atribuição de cópia definido explicitamente pelo programador, então ela muito provavelmente precisa de todos os três.”

- Evitar cópias rasas (*shallow copies*)
- Ao criar uma cópia de um membro alocado dinamicamente, seu ponteiro é copiado e não o conteúdo em memória
 - O que esperar do destrutor e como desalocar memória?

Regra dos Três



```
class CaixaDeInts {
public:
    int* dados; // Ponteiro para os dados

    CaixaDeInts(int valor) {
        dados = new int; // Aloca memória
        *dados = valor;
    }

    ~CaixaDeInts() {
        delete dados; // Libera a memória
    }
    // O compilador gera o construtor de cópia e o operator= aqui!
};

int main() {
    CaixaDeInts caixa1(10);
    CaixaDeInts caixa2 = caixa1;
}
```

```
$ ./copia
free(): double free detected in tcache 2
Aborted
```


Regra dos Três



```
class CaixaDeInts {
public:
    // ...

    CaixaDeInts(const CaixaDeInts& outra) {
        dados = new int(*outra.dados);
    }

    CaixaDeInts& operator=(const CaixaDeInts& outra) {
        if (this != &outra) {
            delete dados;
            dados = new int(*outra.dados);
        }
        return *this;
    }
};
```

- Permita a cópia profunda
 - Copia todos os dados em memória e não o ponteiro

Exemplos



Operadores aritméticos e relacionais

```
// funções globais

double operator*(const Ponto3D& a, const Ponto3D& b) {
    return a.x*b.x + a.y*b.y + a.z*b.z;
}

Ponto3D operator*(const Ponto3D& p, double escalar) {
    return Ponto3D(p.x * escalar, p.y * escalar, p.z * escalar);
}

Ponto3D operator/(const Ponto3D& p, double escalar) {
    return Ponto3D(p.x / escalar, p.y / escalar, p.z / escalar);
}

bool operator==(const Ponto3D& a, const Ponto3D& b) {
    return (a.x == b.x && a.y == b.y && a.z == b.z);
}

bool operator!=(const Ponto3D& a, const Ponto3D& b) {
    return !(a == b);
}

// ...
```

Exemplos



Operadores de redirecionamento de entrada/saída

- Podemos sobrecarregar os operadores >> e << para receber e imprimir dados membro no terminal
- Recebem e retornam streams de I/O como parâmetro (istream e ostream)

```
std::ostream& operator<<(std::ostream& os, const Ponto3D& p) {  
    os << "(" << p.x << ", " << p.y << ", " << p.z << ")";  
    return os;  
}  
  
std::istream& operator>>(std::istream& is, Ponto3D& p) {  
    std::cout << "Digite x y z: ";  
    is >> p.x >> p.y >> p.z;  
    return is;  
}
```

Exemplos

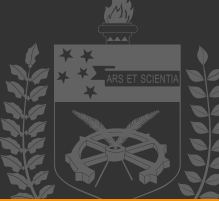


Operadores de função

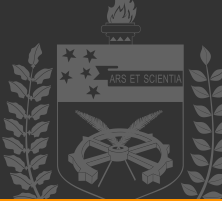
- É possível sobrecarregar o operador (), permitindo que objetos sejam chamados como se fossem funções
- Objetos com operator() são chamados de functors, ou *function objects*

```
class Ponto3D {  
    public:  
    // ... membros da classe Ponto3D  
  
    void operator()(double _x, double _y, double _z) {  
        x = _x; y = _y; z = _z;  
    }  
  
    double operator()(const Ponto3D& other) const {  
        return std::sqrt(  
            (x - other.x)*(x - other.x) +  
            (y - other.y)*(y - other.y) +  
            (z - other.z)*(z - other.z)  
        );  
    }  
}
```

Operadores não sobrecarregáveis

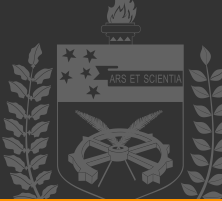


- Para preservar alguns aspectos básicos da linguagem e a previsibilidade do comportamento, alguns operadores não são sobrecarregáveis, são eles:
- Operadores de acesso a membros $\rightarrow a.b$ e $a.*b$
- Operador de resolução de escopo $\rightarrow a::b$
- Operador condicional ternário $\rightarrow c ? a : b$
- Operador de tamanho $\rightarrow \text{sizeof}(a)$
- Também não é possível criar novos operadores



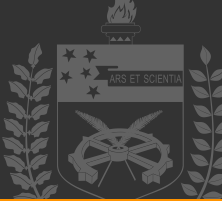
- Implementamos algumas das sobrecargas como funções membro da classe Ponto3D e outras como funções globais
- Apenas funciona pois x, y e z são atributos públicos da classe
 - Quebra do encapsulamento dos dados
- Caso as coordenadas se tornem atributos privados, os operadores não possuem acesso direto
- Podemos fornecer acesso aos membros privados e protegidos de uma classe declarando uma relação de amizade

Funções amigas



- Para declarar uma função amiga, use a palavra-chave `friend` dentro da classe
- Implementação da função fora do escopo da classe

```
class Ponto3D {  
    private:  
        double x, y, z;  
    public:  
        Ponto3D(double _x = 0, double _y = 0, double _z = 0) :  
            x(_x), y(_y), z(_z) {}  
  
        friend Ponto3D operator+(const Ponto3D& a, const Ponto3D& b);  
};  
  
Ponto3D operator+(const Ponto3D& a, const Ponto3D& b) {  
    return Ponto3D(a.x + b.x, a.y + b.y, a.z + b.z);  
}
```



Implementamos algumas das sobrecargas como funções membro da classe Ponto3D e outras como funções globais

- Quando usar funções membros ou globais?
 - Operadores unários → implemente como função membro da classe
 - Operadores binários simétricos → implemente como função global
 - Operadores binários assimétricos → implemente como função membro, especialmente se precisar acessar membros privados
 - Estas regras podem possuir algumas raras exceções, como os operadores >> e <<, que são comumente implementados como funções globais amigas



- Assim como funções, classes inteiras podem ser declaradas como friend
- Uma classe amiga tem acesso direto a membros privados e protegidos da classe que a declarou
- Útil quando duas classes têm forte relação conceitual
 - Deve ser usado com cuidado, pois fere o encapsulamento
- Relação não comutativa
 - Classe A amiga de B $\neq$ Classe B amiga de A

Classes amigas



```
class ClasseA {
private:
    int valor;
public:
    ClasseA(int v) : valor(v) {}

    // ClasseB terá acesso direto a membros privados de ClasseA
    friend class ClasseB;
};

class ClasseB {
public:
    void mostrar(const ClasseA& a) {
        std::cout << "Valor = " << a.valor << "\n";
    }
};
```

- B é amiga de A, mas não o contrário
- B acessa os membros privados de A, mas A não pode acessar os membros privados de B



- Crie uma classe Tempo com atributos horas, minutos e segundos e sobrecarregue os seguintes operadores:
 1. Operadores `=`, `+=`, `-=`, `[]` usando `0` $\rightarrow$ horas; `1` $\rightarrow$ minutos; `2` $\rightarrow$ segundos
 2. Operadores `+`, `-`, `==` e `!=` para soma, subtração e comparação de objetos
 3. Operadores `>>` e `<<` para imprimir e ler valores no formato `hh:mm:ss`
 4. Operador `()` para atualizar o tempo com base em três valores